

Bloxorz 3D

Start with the identity of the game. It is a Bloxorz-style 3D puzzle where the player rolls a cuboid across a tile map until it stands on the goal. Mention Three.js and WebGL, then move quickly into what was implemented.

What was built

Say that this is not just one level. The project has eight official levels, saved progress, a level editor, fragile tiles, switches, bridges, teleport split mode, and verified par values. Keep this slide broad and save the rule details for the mechanics slide.

Discrete game state

The important implementation point is that the puzzle is stored as data, not guessed from the mesh. The state is x , z , and orientation. Standing occupies one cell, lying occupies two cells, and those occupied cells decide movement, falling, switches, and winning.

Movement and animation

A key press first changes the exact grid state. The mesh then animates toward that state. Position interpolation makes the movement smooth, quaternion rotation gives clean rolling, and snapping after the move prevents floating-point drift.

Level mechanics

Explain that each tile type changes validation. Fragile tiles punish standing upright, heavy switches require upright pressure, soft switches react to any occupied cell, bridges change the board, teleport splits the block, and the goal only accepts a standing full block.

Graphics pipeline

Connect this directly to lectures. The implementation starts from geometry, applies model transforms, uses the camera as the view transform, applies perspective projection, rasterizes triangles, shades fragments with materials and lights, and writes the final result to the canvas.

Scene setup

Say clearly that the project uses library-managed shaders, not custom block GLSL. Three.js handles camera, materials, lights, shadows, fog, and the WebGL renderer. The postprocessing package handles SMAA, bloom, and vignette. The editor uses raycasting for mouse placement.

Player input

The same game state drives keyboard controls, touch controls, the third-person camera, the HUD, hints, and saved progress. This is a useful point because it shows the implementation is not separate hacks for desktop, mobile, and camera behavior.

BFS solver

BFS means breadth-first search. It explores all one-move states, then all two-move states, then all three-move states. Because of this order, the first solution found is the shortest solution.

The state includes position, orientation, and bridge state. The hint uses the next move from BFS. The check-levels script verifies all official par values, including the split level.

Code files

If code is requested, start with `controls.js` and `grid.js`. `controls.js` explains movement and validation. `grid.js` explains tiles and bridges. `solver.js` explains BFS. `editor.js` explains raycasting. `scene`, `camera`, and `lighting` files explain the graphics setup.

Demo plan

Keep the live demo focused. Show level selection, Level 1 movement, one fragile tile, one switch and bridge, teleport split and merge, one BFS hint, and the editor only if time remains. Do not spend too long solving a full level live.